

Slooze Food Ordering

Backend API Documentation

NestJS · GraphQL · Prisma · PostgreSQL

RBAC

Re-BAC

JWT Auth

GraphQL

Vercel Ready

v1.0 · Take-Home Assignment Submission

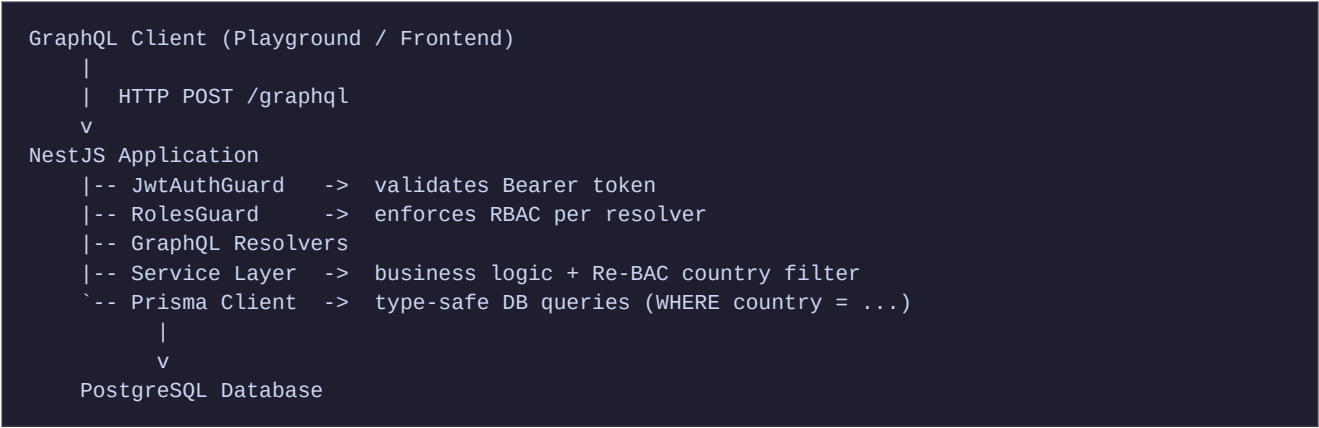
Table of Contents

- 1. Project Overview & Architecture
- 2. Tech Stack
- 3. Database Schema & Entity Relationships
- 4. Authentication & Authorization
- 5. API Reference — GraphQL Operations
- 6. Seeded Dataset
- 7. Local Setup & Running the App
- 8. Testing
- 9. Deployment on Vercel

1. Project Overview & Architecture

A production-ready NestJS backend for a role-based food ordering platform. Users are organised into three roles — Admin, Manager, and Member — each with distinct permissions. An additional country-based access layer (Re-BAC) ensures that Managers and Members can only view and act on data within their assigned country.

System Architecture



Access Control Model

Feature	Admin	Manager	Member
View restaurants & menu items	Yes - All countries	Yes - Own country	Yes - Own country
Create order / add items	Yes	Yes	Yes

Feature	Admin	Manager	Member
Place order (checkout & pay)	Yes	Yes	No
Cancel order	Yes	Yes	No
Modify payment methods	Yes	No	No

2. Tech Stack

Technology	Version	Purpose
NestJS	latest	Backend framework
GraphQL	code-first	API layer via Apollo Server
Prisma ORM	latest	Type-safe database access
PostgreSQL	15+	Primary database
JWT / Passport	latest	Authentication & strategy
bcrypt	latest	Password hashing (salt rounds: 10)
TypeScript	5+	Programming language
Jest	latest	Unit & E2E testing
Vercel	v2	Cloud deployment

3. Database Schema & Entity Relationships

Entity Relationship Overview

User	-----+-----	Orders (1:N)
	`-----	PaymentMethods (1:N)
Restaurant	-----+-----	MenuItems (1:N)
	`-----	Orders (1:N)
Order	-----+-----	OrderItems (1:N)
	-----	User (N:1)
	-----	Restaurant (N:1)
	`-----	PaymentMethod (N:1)
OrderItem	-----+-----	Order (N:1)
	`-----	MenuItem (N:1)

Schema Models

User

Field	Type	Description
id	String	UUID primary key
name	String	Display name

Field	Type	Description
email	String	Unique login email
passwordHash	String	bcrypt hash – never exposed
role	Role	ADMIN MANAGER MEMBER
country	Country?	INDIA AMERICA null (Admin)
createdAt / updatedAt	DateTime	Auto-managed timestamps

Restaurant

Field	Type	Description
id	String	UUID primary key
name	String	Restaurant name
description	String	Short description
country	Country	INDIA AMERICA
isActive	Boolean	Soft enable/disable
menuItems	MenuItem[]	Relation

MenuItem

Field	Type	Description
id	String	UUID primary key
name	String	Dish name
price	Float	Price in local currency
isAvailable	Boolean	Stock flag
restaurantId	String	FK → Restaurant

Order

Field	Type	Description
id	String	UUID primary key
status	OrderStatus	PENDING PLACED CANCELLED
userId	String	FK → User
restaurantId	String	FK → Restaurant
totalAmount	Float	Sum of orderItems
orderItems	OrderItem[]	Relation

PaymentMethod

Field	Type	Description
id	String	UUID primary key
type	PaymentType	CARD UPI WALLET
label	String	e.g. Visa ending 4242
isDefault	Boolean	Default payment flag
userId	String	FK → User (Admin only)

4. Authentication & Authorization

Authentication Flow

1. Client calls `login(email, password)` mutation
2. Server fetches user by email from DB
3. `bcrypt.compare(password, passwordHash)`
4. On success → JWT signed with `{ sub, role, country }` (expires: 7d)
5. Client stores token and sends on every request:
`Authorization: Bearer <JWT_TOKEN>`
6. `JwtAuthGuard` validates token on every protected resolver
7. `@CurrentUser()` decorator injects the decoded user into resolver context

RBAC — Role-Based Access Control

Implemented via a custom `@Roles()` decorator and `RolesGuard`. Guards are applied at the resolver level. A `ForbiddenException` is thrown immediately when the role does not match, before any service logic runs.

Re-BAC — Country-Based Access Control

Implemented inside the service layer at the **Prisma query level** (not post-fetch filtering). Non-admin users automatically get a WHERE clause injected based on their JWT country. Accessing another country's resource returns: *ForbiddenException: "Access denied: country restriction"*.

Role	Country Filter Applied
ADMIN	None – full global access
MANAGER	WHERE country = user.country
MEMBER	WHERE country = user.country

5. API Reference — GraphQL Operations

All operations are served at **POST /graphql**. Protected operations require **Authorization: Bearer <token>** header. The GraphQL Playground is available at **GET /graphql**.

5.1 Auth Mutations (public)

Register

```
mutation {
  register(
    name: "Tony Stark"
    email: "tony@shield.gov"
    password: "password123"
    role: MEMBER
    country: AMERICA
  ) {
    token
    user { id email role country }
  }
}
```

Login

```

mutation {
  login(
    email: "captainmarvel@shield.gov"
    password: "password123"
  ) {
    token
    user { id name role country }
  }
}

```

5.2 Restaurant Queries (all roles, country-filtered)

List Restaurants

```

query {
  restaurants {
    id
    name
    country
    isActive
    menuItems {
      id name price isAvailable
    }
  }
}

```

Get Single Restaurant

```

query {
  restaurant(id: "restaurant-uuid") {
    id name description country
    menuItems { id name price }
  }
}

```

Get Menu Items

```

query {
  menuItems(restaurantId: "restaurant-uuid") {
    id name price isAvailable
  }
}

```

5.3 Order Operations

Mutation / Query	Allowed Roles	Description
createOrder(restaurantId)	ALL	Start a new PENDING order
addItemToOrder(orderId, menuItemId, qty)	ALL	Add/increase item in order
removeItemFromOrder(orderId, orderItemId)	ALL	Remove item from order
placeOrder(orderId, paymentMethodId)	ADMIN, MANAGER	Checkout – sets status PLACED
cancelOrder(orderId)	ADMIN, MANAGER	Cancel – sets status CANCELLED
myOrders	ALL	Fetch own orders

Mutation / Query	Allowed Roles	Description
order(id)	ALL	Fetch single own order

Create Order

```
mutation {
  createOrder(restaurantId: "restaurant-uuid") {
    id status totalAmount createdAt
  }
}
```

Add Item to Order

```
mutation {
  addItemToOrder(
    orderId: "order-uuid"
    menuItemId: "menu-item-uuid"
    qty: 2
  ) {
    id
    totalAmount
    orderItems {
      id quantity unitPrice
      menuItem { name price }
    }
  }
}
```

Remove Item from Order

```
mutation {
  removeItemFromOrder(
    orderId: "order-uuid"
    orderItemId: "order-item-uuid"
  ) {
    id totalAmount
    orderItems { id quantity menuItem { name } }
  }
}
```

Place Order (ADMIN / MANAGER only)

```
mutation {
  placeOrder(
    orderId: "order-uuid"
    paymentMethodId: "payment-method-uuid"
  ) {
    id status totalAmount
  }
}
```

Cancel Order (ADMIN / MANAGER only)

```
mutation {
  cancelOrder(orderId: "order-uuid") {
    id status
  }
}
```

My Orders

```
query {
  myOrders {
    id status totalAmount createdAt
    restaurant { name country }
    orderItems {
      quantity unitPrice
      menuItem { name }
    }
  }
}
```

5.4 Payment Method Operations (ADMIN only)

List My Payment Methods

```
query {
  myPaymentMethods {
    id type label isDefault
  }
}
```

Add Payment Method

```
mutation {
  addPaymentMethod(
    type: CARD
    label: "Visa ending 9999"
  ) {
    id type label isDefault
  }
}
```

Update Payment Method

```
mutation {
  updatePaymentMethod(
    id: "payment-method-uuid"
    label: "Mastercard ending 1234"
    isDefault: true
  ) {
    id type label isDefault
  }
}
```

Delete Payment Method

```
mutation {
  deletePaymentMethod(id: "payment-method-uuid")
}
```


5.5 Common Error Responses

Scenario	Error Message
No / invalid JWT token	Unauthorized
Role not permitted	Forbidden resource
Cross-country access attempt	Access denied: country restriction
Wrong password	Invalid credentials
Duplicate email on register	Email already in use
Order not found / not yours	Order not found
MEMBER tries to place order	Forbidden resource
MANAGER tries to add payment	Forbidden resource

6. Seeded Dataset

Users (all passwords: password123)

Name	Email	Role	Country
Nick Fury	nickfury@shield.gov	ADMIN	Global (null)
Captain Marvel	captainmarvel@shield.gov	MANAGER	INDIA
Captain America	captainamerica@shield.gov	MANAGER	AMERICA
Thanos	thanos@shield.gov	MEMBER	INDIA
Thor	thor@shield.gov	MEMBER	INDIA
Travis	travis@shield.gov	MEMBER	AMERICA

Restaurants

Name	Country	Sample Menu Items
Spice Garden	INDIA	Butter Chicken, Dal Makhani, Biryani, Paneer Tikka
Mumbai Bites	INDIA	Vada Pav, Pav Bhaji, Misal, Bhel Puri
Delhi Darbar	INDIA	Chole Bhature, Rajma Rice, Aloo Paratha, Lassi
Burger Palace	AMERICA	Classic Burger, BBQ Bacon Burger, Fries, Shake
NY Pizza Co	AMERICA	Margherita, Pepperoni, BBQ Chicken, Garlic Bread
Texas BBQ House	AMERICA	Brisket, Pulled Pork, Ribs, Coleslaw

Nick Fury's Payment Methods

Type	Label	Default
CARD	Visa ending 4242	Yes
UPI	nickfury@upi	No
WALLET	Amazon Pay	No

7. Local Setup & Running the App

Prerequisites

- Node.js 20+
- PostgreSQL 15+
- npm

Step-by-Step Setup

Step 1: Clone the repository

```
git clone <your-github-url>
cd slooze-backend
```

Step 2: Install dependencies

```
npm install
```

Step 3: Configure environment

```
cp .env.example .env
# Edit .env and set:
# DATABASE_URL=postgresql://user:pass@localhost:5432/slooze
# JWT_SECRET=your-super-secret-key
# PORT=3000
```

Step 4: Generate Prisma Client

```
npx prisma generate
```

Step 5: Push schema to database

```
npx prisma db push
```

Step 6: Seed the database

```
npm run seed
```

Step 7: Start development server

```
npm run start:dev
```

Step 8: Open GraphQL Playground

```
http://localhost:3000/graphql
```

Environment Variables Reference

Variable	Example Value	Description
DATABASE_URL	postgresql://user:pass@host:5432/slooze	PostgreSQL connection string
JWT_SECRET	your-super-secret-key	JWT signing secret
PORT	3000	Server port (optional)
NODE_ENV	production	Set to production on Vercel

8. Testing

Run Unit Tests

```
npm run test
```

Run E2E Tests

```
npm run test:e2e
```

Test Coverage Areas

Test File	Scenarios Covered
auth.service.spec.ts	Register, duplicate email, login, wrong password
restaurants.service.spec.ts	ADMIN sees all; MANAGER/MEMBER filtered by country
orders.service.spec.ts	MEMBER blocked from place/cancel; MANAGER allowed; cross-user blocked
payments.service.spec.ts	Only ADMIN can add/update/delete payment methods
rebac.spec.ts	Indian Manager blocked from US data and vice versa; Admin unrestricted
app.e2e-spec.ts	Full happy path: login → createOrder → addItem → placeOrder

9. Deployment on Vercel

vercel.json

```
{
  "version": 2,
  "builds": [{ "src": "dist/main.js", "use": "@vercel/node" }],
  "routes": [{ "src": "/(.*)", "dest": "dist/main.js" }]
}
```

Deployment Steps

Push to GitHub

```
git push origin main
```

Install Vercel CLI (optional)

```
npm i -g vercel
```

Deploy via CLI

```
vercel --prod
```

Or connect GitHub repo at

```
https://vercel.com/new
```

Required Vercel Environment Variables

Variable	Value
DATABASE_URL	postgresql://... (use Neon or Supabase for hosted Postgres)
JWT_SECRET	your-production-secret

Variable	Value
NODE_ENV	production

***Tip:** Use **Neon** (neon.tech) or **Supabase** for a free hosted PostgreSQL database that works out-of-the-box with Vercel. After adding environment variables, run **npm run seed** once against the production DATABASE_URL to populate data.*